Detection Time vs. False Alerts in Multi-Window Burn-Rate Alerting

Suman Babu

Abstract

Multi-window, multi-burn-rate alerting is the default way to page on service-level objectives (SLOs), yet its window lengths and thresholds are almost always copied from the Google SRE Workbook rather than derived. We model request errors as a thinned Poisson process and derive closed-form expressions for (i) detection latency as a function of incident severity, window length, threshold, and traffic shift, (ii) the error budget consumed before an alert fires, (iii) reset time after recovery, and (iv) an exponential bound on the per-evaluation false-alert probability,

$$P \le exp\big(- \lambda W \cdot KL(\theta \parallel p_0)\big).$$

The analysis reveals a sharp regime change. Above roughly 0.03 req/s, the standard 1 h/5 m rule is noise-free and its latency is window-limited. Below roughly 0.01 req/s (about 860 requests per day), a single failed request trips both windows, so the alert degenerates into “page on every error.” No re-tuning of windows escapes this: any rule with a bounded false-page rate must wait for k errors, which sets a latency floor of $k/(\lambda p_1)$.

Monte Carlo simulation confirms the latency formula to within evaluation-interval jitter (0.5 min) and reproduces the predicted low-traffic false-page counts. A retrospective check against six alert transitions from three fault-injection drills on a three-node Kubernetes home lab (Project 1) places all six inside the predicted intervals. We close with a design procedure that picks windows, thresholds, and a minimum-error-count guard from explicit targets, and a pre-registered fault-injection protocol for prospective validation.

Keywords: service-level objectives, error budgets, burn-rate alerting, multi-window alerts, detection latency, false alarms, Poisson processes, Chernoff bound, low-traffic services, site reliability engineering, Kubernetes.

Introduction

An SLO alert has two jobs that pull against each other: page quickly when the error budget is genuinely at risk, and stay silent otherwise. Burn-rate alerting resolves this by comparing the observed error ratio over a window W with a multiple B of the budgeted ratio $\varepsilon = 1 - SLO$. The multi-window variant adds a short window that must also exceed the threshold, so the alert clears soon after recovery.

Practitioners almost universally adopt the Workbook’s defaults: $B = 14.4$ over 1 h and 5 m to page, $B = 6$ over 6 h and 30 m to ticket. Those numbers encode a policy (page when 2% of a 30-day budget burns in an hour) but say nothing about how fast a given incident is caught, how much budget is gone by then, or how often the rule fires on noise. They also implicitly assume enough traffic that a window’s error ratio is a stable

estimate. For low-traffic services that assumption fails, and operators report pages on single failed requests.

Research questions

RQ1 (Latency).
For an incident with burn rate b, how long until a multi-window rule with windows (W_L, W_S) and threshold B fires, including pipeline delays?

RQ2 (Noise).
For a healthy service with baseline burn b_0 and request rate λ, how often does the rule fire falsely?

RQ3 (Low traffic).
At what traffic does the standard configuration break, and what is the best achievable latency at a given false-alert budget below that point?

RQ4 (Design).
How should windows, thresholds, and guards be chosen from explicit targets instead of copied?

Contributions

- A closed-form fluid model of detection latency, budget-at-detection, and reset time that accounts for baseline errors and traffic shifts across the incident boundary (§4.1).
- A Chernoff bound on per-evaluation false-alert probability whose exponent is $\lambda W \cdot KL(\theta \parallel p_0)$, giving the minimum requests per window for a target noise level (§4.2).
- A characterisation of the degenerate low-traffic regime and a detection–noise frontier: latency $\geq k/(\lambda p_1)$ for any rule bounded to a false-page budget (§4.3).
- A target-driven design procedure, including a minimum-error-count guard and a “resolvable SLO” traffic condition (§5).
- Monte Carlo verification (§6), a retrospective check against Project 1 drill data, and a pre-registered prospective protocol (§7).

Scope

This paper treats request-based availability and latency SLIs evaluated as ratios over sliding windows, as in Prometheus recording rules. Time-slice (“bad minutes”) SLIs, anomaly detection, and multi-service aggregation are out of scope except where noted in §8.

Background and Related Work

SLOs and error budgets.

An SLO with target $1 - \varepsilon$ over a compliance period T (here 30 days) permits a fraction ε of requests to fail. At rate λ the budget is $\varepsilon\lambda T$ failed requests. Burn rate b is the observed error ratio divided by ε, so $b = 1$ exhausts the budget exactly at the end of T, and $b = 14.4$ exhausts 2% of it per hour .

Burn-rate alert families.

The Workbook walks through six alerting strategies of increasing sophistication, ending with multi-window, multi-burn-rate rules. It argues qualitatively about precision, recall, detection time, and reset time, and tabulates detection time for a 100% outage. It also flags low-traffic services as a problem—suggesting synthetic traffic, aggregation, or changing the SLO—without quantifying when the problem starts. This paper makes those trade-offs quantitative.

Sequential change detection.

Detecting a shift in a Bernoulli failure rate is a classical problem. Shewhart charts threshold a fixed-sample statistic; CUSUM is optimal in Lorden's minimax sense for the trade-off between average run length to false alarm and worst-case detection delay . A sliding-window ratio is a finite-memory relative of both. The literature gives the optimal detector; it doesn't analyze the windowed-ratio rule that operators actually run, nor its interaction with scrape and evaluation pipelines.

Large deviations.

The false-alert bound in §4.2 is a Chernoff bound on a difference of independent Poisson variables. Its exponent reduces to the Kullback–Leibler divergence between threshold and baseline error ratios, which links alert noise directly to information-theoretic distinguishability.

Gap.

No published work, to our knowledge, gives closed-form latency and noise expressions for the multi-window ratio rule, characterises the traffic level at which it degenerates, or turns these into a selection procedure. Practitioner sources treat low-traffic alerting as folklore.

System Model

Notation

Table 1 lists symbols and their Project 1 values.

Notation and Project 1 values.

Symbol	Meaning	Project 1 value
λ	request rate (req/s)	≈ 10
p_0	baseline failure probability	0.001
p_1	incident failure probability	varies
ε	SLO error budget fraction	0.005
T	compliance period	30 days
b	burn rate ($= p/\varepsilon$)	varies
b_0	healthy burn rate	0.2
B	alert threshold (burn-rate multiple)	14.4 / 6
W_L, W_S	long and short windows	1 h, 5 m / 6 h, 30 m
θ	threshold ratio ($= B\varepsilon$)	0.072 / 0.030
D	pipeline delay (scrape + eval + ConfigMap)	60–90 s
Δ	evaluation interval	1 min
F	pending duration (`for:`)	0
ρ	traffic ratio after/before incident	1 (assumed)
g	fraction of window with missing pre-incident traffic	0
k	minimum error count guard	3 (recommended)
C^*	budget fraction triggering a page	2%
Φ^*	false-page target per period	1/month

Error process

Base model (thinned Poisson).

Requests arrive as a Poisson process of rate $\lambda(t)$. Each request fails independently with probability $p(t)$. Failures and successes are then independent Poisson processes of

rates λp and $\lambda(1-p)$. An incident is a step: $p = p_0$ for $t < 0$ and $p = p_1$ for $t \geq 0$, with traffic λ_0 before and λ_1 after.

Bursty extension (compound Poisson).

Real failures cluster: a retry storm or a dying connection fails several requests at once. We model failure events as Poisson with clusters of mean size m and second moment m_2. The index of dispersion of the error count rises from 1 to m_2/m, and §4.2 shows this divides the noise exponent by roughly that factor. A Markov-modulated variant (healthy and degraded states with exponential sojourns) is used in §7 to model flapping backends.

Estimator and rule

For window W, let $N_W(t)$ and $E_W(t)$ count requests and failures in $(t-W,\, t]$. The estimator is the ratio $r_W(t) = E_W(t)/N_W(t)$, undefined when $N_W = 0$. This matches `increase(errors[W]) / increase(requests[W])` in Prometheus, where `0/0` yields `NaN` and any comparison with `NaN` is false.

The multi-window rule is evaluated every Δ and fires when both windows exceed the threshold:

$$A(t) = \left[r_{W_L}(t) > B\varepsilon\right] \wedge \left[r_{W_S}(t) > B\varepsilon\right].$$

An optional pending duration F (`for:`) requires A to hold continuously before firing. An optional guard requires $E_{W_L} \geq k$ (§4.3). The quantities of interest are detection latency L (incident onset to A first true), budget consumed at detection C, reset time R (recovery to A false), and false-alert rate Φ (alert episodes per T while healthy).

Analysis

The central result is that the product $B \cdot W_L$ alone fixes how fast an incident of any severity is caught and how much budget it burns first; the split between B and W_L only decides the weakest detectable incident and how noisy the rule is.

Detection latency (fluid limit)

Replace counts by their means. At time $t \in [0, W]$ after onset, the window holds $\lambda_1 t$ incident requests and $\lambda_0(W-t)$ pre-incident ones. Setting $r_W(t) = \theta$ and solving gives the window's crossing time:

$$t_W = \frac{W\left(\theta - b_0 \varepsilon \lambda_0/\lambda\right)}{\rho\, p_1 - b_0 \varepsilon(\rho-1)\lambda_0/\lambda - \theta}.$$

With steady traffic and a clean baseline ($\rho = 1$, $b_0 = 0$) this reduces to $t_W = W \cdot B/b$. Because t_W scales with W, the long window always crosses last, and the end-to-end latency is:

$$L = t_{W_L} + D + \frac{\Delta}{2} + F,$$

where D is pipeline delay (fault visible to metrics, scrape, recording-rule evaluation), $\Delta/2$ is the expected phase of the alert evaluation, and F is the pending duration.

Budget consumed at detection.

Multiplying burn rate by latency and dividing by T:

$$C = \frac{b \cdot L}{T} = \underbrace{\frac{B \cdot W_L}{T}}_{\hookrightarrow \textit{policy term}} + \underbrace{\frac{b\,(D+\Delta/2+F)}{T}}_{\hookrightarrow \textit{pipeline term}}.$$

The policy term is independent of severity. For the Workbook page rule it is $14.4 \times 1\,h / 720\,h = 2.0\%$; for the ticket rule, $6 \times 6\,h / 720\,h = 5.0\%$. This is exactly the Workbook's design intent, now derived rather than asserted. The pipeline term grows with b: a total outage of a 99.5% service ($b = 200$) with $D + \Delta/2 + F = 3$ min adds 1.4% of budget before anyone is paged.

Since $b \cdot t_{W_L} = B \cdot W_L$, any two configurations with the same product detect every incident equally fast. Halving W_L and doubling B buys nothing in speed. It raises the weakest detectable burn rate (b must exceed B) and, as §4.2 shows, changes noise.

Traffic shifts.

If clients back off during an incident ($\rho > 1$), the window stays diluted by older healthy traffic and detection slows; with $\rho = 1.4$, $b = 100$, and $B = 14.4$, t_{W_L} grows from 8.6 to 11.4 min. A missing stretch of pre-incident traffic acts the other way: if a fraction g of the window's expected requests is absent, $t_W \approx (1 - g) \cdot W \cdot B/b$.

Reset time.

For an incident lasting at least W_S, the short window drops below θ after the faulty share of it falls to B/b:

$$R = W_S\left(1 - \frac{B}{b}\right).$$

Stochastic jitter.

Near the crossing, r_W has standard deviation $\sqrt{\theta(1-\theta)/(\lambda W)}$ and slope $(p_1 - p_0)/W$, so latency jitter is $\sigma_L \approx \sqrt{\theta(1-\theta)W/\lambda}\,/\,(p_1 - p_0)$. At $\lambda = 10$, $W = 1$ h, $p_1 = 0.5$ this is 10 s, smaller than Δ. At $\lambda = 0.01$ it is about 5 min and the fluid limit stops being a good guide (§4.3).

False alerts

In a healthy window, $E \sim Poisson(\lambda W p_0)$ and the successes $S \sim Poisson(\lambda W(1-p_0))$ are independent. The window fires when $(1-\theta)E - \theta S > 0$. Optimizing the Chernoff bound over the exponential tilt gives a closed form:

$$P \le exp\Big(-\lambda W \cdot KL(\theta \parallel p_0)\Big),$$

with $KL(\theta \parallel p_0) = \theta ln(\theta/p_0) + (1-\theta) ln\Big((1-\theta)/(1-p_0)\Big)$. Noise therefore decays exponentially in the expected number of requests per window, at a rate set by how distinguishable the threshold is from the baseline.

Minimum requests per window.

For a per-evaluation false-fire probability δ, a window needs at least

$$n^* = \frac{ln(1/\delta)}{KL(\theta \| p_0)}$$

requests. For the page threshold $\theta = 0.072$ with $b_0 = 0.2$ ($p_0 = 0.001$), $KL = 0.213$; $\delta = 10^{-6}$ needs $n^* \approx 65$ requests. A 5-minute window meets that above 0.22 req/s; a 1-hour window above 0.018 req/s.

Multi-window rule.

The AND rule fires only if both windows do, so $P(A) \le min(P_L, P_S)$, and the long window—holding 12 times the requests—normally dominates. An upper bound on episodes per period is $\Phi \le (T/\Delta) \cdot min(P_L, P_S)$. Consecutive evaluations are strongly correlated, so this bound is loose; §6 measures Φ directly.

Bursty errors.

Under compound-Poisson failures with dispersion index $d = m_2/m$, a Gaussian argument scales the exponent by roughly $1/d$. Clusters of 5 correlated failures need about 5 times as many requests per window for the same noise level.

The low-traffic regime

The ratio estimator has resolution $1/N_W$. When a window holds fewer than $1/\theta$ requests, one failure alone exceeds the threshold. In expectation this happens when:

$$\lambda < \frac{1}{\theta W}.$$

Traffic below which one failure trips each Workbook window.

Window	θ	λ_{deg} (req/s)
Long ($W_L = 1$ h, page)	0.072	0.0039
Short ($W_S = 5$ m, page)	0.072	0.046
Long ($W_L = 6$ h, ticket)	0.030	0.0015
Short ($W_S = 30$ m, ticket)	0.030	0.018

Between 0.0039 and 0.046 req/s the short window is degenerate and only the long window stands between a single error and a page. Below 0.0039 req/s both are, and the page rule fires on almost every healthy failure: $\Phi \approx \lambda p_0 T \cdot Pr(N_{W_L} < 1/\theta)$. At $\lambda = 0.001$ req/s and $b_0 = 0.2$ that is about 2.6 pages a month while the service uses 20% of its budget.

The page is not wrong; the SLO is unresolvable. At $\lambda = 0.001$ req/s the entire monthly budget is $\varepsilon\lambda T = 13$ failures, so one failure is 7.7% of budget, more than the 2% the page rule is meant to catch. No re-tuning of windows fixes this. The SLO can only be held to the granularity of individual requests if:

$$\lambda \geq \frac{k}{C^* \cdot \varepsilon \cdot T},$$

where C^* is the budget fraction that should trigger a page and k is the error count that can be trusted as signal. With $k = 3$, $C^* = 2\%$, $\varepsilon = 0.005$, $T = 30$ days, the service needs $\lambda \geq 0.0116$ req/s, about 1,000 requests a day.

Detection–noise frontier.

Any rule that holds false pages to Φ^* per period must wait for some k errors, where k is the smallest integer with $(T/W_L) \cdot Pr\{Poisson(\lambda p_0 W_L) \geq k\} \leq \Phi^*$. Waiting for k failures at rate λp_1 takes an $Erlang(k, \lambda p_1)$ time, so:

$$L_{floor} = \frac{k}{\lambda p_1}.$$

Above the crossover traffic $\lambda_\times = k \cdot b / (B \cdot W_L \cdot p_1)$ latency is window-limited and the Workbook defaults are fine. Below it latency is traffic-limited: it scales as $1/\lambda$ and no choice of window helps. For the page rule with $k = 3$ and a 50% error incident, $\lambda_\times = 0.012$ req/s.

Latency SLIs

Everything above applies to latency SLIs with “slower than the threshold” in place of “failed.” Two differences matter in practice. Fault injection that adds a fixed delay makes $p_1 = 1$, so latency is minimal at $W_L \cdot B\varepsilon$. And the SLI is computed from histogram buckets, so the threshold must sit on a bucket boundary or the estimator is biased.

Design Method

Windows and thresholds should be derived from four targets, in this order, rather than copied. Each step uses one result from §4.

Inputs.

ε and T from the SLO; λ_{min}, the lowest sustained request rate (use the overnight trough, not the mean); b_0, the healthy burn rate from 30 days of history; D, measured by fault injection (§7); and targets C^* (budget that may burn before paging), b_{min} (weakest incident that must page), L^* at severity b^* (latency target), and Φ^* (false pages per period).

Step 1: fix the product from budget.

Set $B \cdot W_L = C^* \cdot T$. For $C^* = 2\%$ and $T = 30$ days, $B \cdot W_L = 14.4$ h. This alone sets latency for every severity: $L \approx C^* \cdot T/b + D$.

Step 2: check latency.

Confirm $C^* \cdot T/b^* + D \le L^*$. If not, C^* must fall—you can’t be both slow to page and fast to detect.

Step 3: split the product for sensitivity.

Choose B just below b_{min} with a margin over b_0 ($B \ge 3b_0$ is a reasonable floor). Then $W_L = C^* \cdot T/B$.

Step 4: check noise.

Require $\lambda_{min} \cdot W_L \geq ln(T/(\Delta \cdot \Phi^*)) / KL(B\varepsilon, b_0\varepsilon)$. If it fails, the service is in or near the low-traffic regime; go to Step 6.

Step 5: pick the short window for reset.

W_S trades reset time $R = W_S(1 - B/b)$ against noise. Choose the smallest W_S whose single-error trip rate is acceptable; $W_S = W_L/12$ is a sensible default only when $\lambda_{min} > \lambda_{deg}(W_S)$.

Step 6: low-traffic fallback.

Add the guard $E_{W_L} \geq k$ with k the smallest integer meeting Φ^*. Then check the resolvable-SLO condition $\varepsilon\lambda T \geq k/C^*$. If it fails, no alert rule can meet the targets: lengthen T, loosen ε, aggregate the service with siblings, or add synthetic probes to raise λ.

Worked examples

All three use the Project 1 availability SLO ($\varepsilon = 0.005$, $T = 30$ days, $b_0 = 0.2$, $\Delta = 1$ min, $\Phi^* = 1$ false page per month, $C^* = 2\%$).

The design procedure applied to three services.

Service	λ_{min}	B	W_L	W_S	Notes
High traffic	10 req/s	14.4	1 h	5 m	Reproduces Workbook defaults
Mid traffic	0.1 req/s	14.4	1 h	5 m	Guard $k = 3$ needed
Low traffic	0.005 req/s	–	–	–	SLO unresolvable; widen or aggregate

The recipe reproduces the Workbook's page rule for a high-traffic service, which is the expected sanity check. Its value lies in the other two rows, where copying the defaults either pages on noise or silently over-promises an SLO that the traffic can't support.

Numerical Verification

Simulation confirms the fluid latency formula to within 0.6 min at production traffic and reproduces the single-error false-page model within 11% at low traffic. We simulated requests per second as Poisson counts with binomial failures, and evaluated rules every 60 s at a random phase, matching the Prometheus ratio semantics of §3.3 (pipeline delay D set to zero).

Latency at $\lambda = 10$ req/s

Detection latency at $\lambda = 10$ req/s: fluid prediction against simulation.

Burn rate b	Fluid t_{W_L} (min)	Simulated median (min)	Residual (min)
200	0.43	0.9	+0.5
50	1.73	2.2	+0.5
14.4	6.0	6.5	+0.5
6	14.4	14.9	+0.5

Fluid values use $b_0 = 0.2$. Once the half-interval evaluation phase is added, the formula matches simulation to 0.1 min at every severity, and the residual spread is the evaluation phase itself.

Per-evaluation false-fire probability

Per-evaluation false-fire probability: exact Poisson–binomial tail against the Chernoff bound.

Requests per window n	Exact P	Bound [eq:chernoff]
10	4.9×10^{-1}	1.0
14	9.2×10^{-2}	9.7×10^{-1}
30	1.1×10^{-3}	7.2×10^{-2}
65	2.3×10^{-7}	8.3×10^{-6}
100	3.1×10^{-10}	1.2×10^{-8}

The bound holds everywhere ($\theta = 0.072$, $p_0 = 0.001$) and its slope matches the exact tail, so it's safe for design; it is conservative by one to two orders of magnitude. The exact probability rises from 5 to 10 requests because single-error trips need $N < 1/\theta \approx 14$, so the bound is weakest in exactly the degenerate regime, where the single-error model takes over.

The low-traffic frontier

Low-traffic frontier of the Workbook page rule ($B = 14.4$, 1 h/5 m windows, $\varepsilon = 0.005$). (A) False pages per month while healthy ($b_0 = 0.2$): simulated against the single-error model. (B) Detection latency for a 50% error incident, median with a line to the 90th percentile, with and without a 3-error guard. Monte Carlo, $\Delta = 1$ min; 24 simulated months per rate below 0.03 req/s, 200 incidents per latency point.

False pages peak near 0.003 req/s, not at the lowest traffic: below that peak there are too few errors to page on, and above it the long window holds enough requests to absorb one. The single-error model ($\lambda p_0 T \cdot Pr(N_{W_L} \leq 13)$) predicts 6.22 against 6.21 simulated at the peak; at 0.006 req/s it predicts 0.52 against 0.96 simulated, because two-error coincidences start to matter there. The guard's latency follows $3/(\lambda p_1)$ below 0.01 req/s, as §4.3 predicts (100 min predicted, 101 min simulated at 0.001 req/s).

Experimental Validation on Project 1

A retrospective check on existing drill data supports the model, and a pre-registered protocol will test it prospectively. Project 1 is a three-node k3s cluster (pc1, pc2, laptop on WSL2) running `slo-demo` with two replicas, Prometheus scraping every 15 s, Grafana alerts to Discord, and fault injection through the `slo-demo-fault` ConfigMap (`error_rate`, `latency_ms`).

Retrospective check

Six alert transitions from Drills 1, 3, and 4 have clean timestamps. Drill 1's ticket is excluded because Prometheus held less than 6 h of history, so its 6 h window wasn't full.

Retrospective check: fluid predictions against Project 1 drill timings.

Drill	Alert tier	Predicted (min)	Observed (min)	In band?
1	Fast page	12.0	12.2	Yes
1	Slow ticket	–	10:39:37 IST	(excluded)
3	Fast page	15.1	15.1	Yes (upper edge)
3	Slow ticket	30.0	29.9	Yes
4	Slow ticket	196	207	Yes (with gap correction)
4	Fast never	–	–	Correct

Retrospective residuals: observed transition time minus fluid-model prediction for six alert transitions in Project 1 Drills 1, 3, and 4 (30 Sep–1 Oct 2026). The shaded band is

the assumed pipeline delay D (60–90 s ConfigMap pickup, 1 min rule evaluation, 15 s scrape); whiskers span the unknown pre-incident traffic ratio in Drill 3. The hollow marker shows Drill 4 without the traffic-gap correction.

The fluid model omits pipeline delay, so the residual (observed minus prediction) should fall inside the band for D. All six fall in or overlap it, though Drill 3's page sits at the band's upper edge.

Two caveats keep this from being a proof. The D band is assumed, not measured, and is wide relative to the short-window events. Drill 3's traffic ratio is uncertain because stray load generators ran for an unknown part of the window, so its two points constrain the model weakly. Drill 4 is the strongest test: it's long enough that D is small relative to the prediction, and only the traffic-gap correction from [eq:crossing] brings it into agreement.

Prospective protocol

The prospective study tests three hypotheses, each with a pass criterion fixed before any run.

H1 (Latency).
Measured latency minus D lies within $\pm\ 2\sigma_L + \Delta$ of t_{W_L} for every cell in the grid below.

H2 (Noise).
At emulated low traffic, false pages per 24 h of healthy run agree with the §6 simulation within a Poisson 95% interval.

H3 (Guard).
With the $E_{W_L} \geq 3$ guard, false pages fall to zero and latency follows $max(B \cdot W_L/b,\ 3/(\lambda p_1)) + D$.

Factors and levels for the prospective fault-injection study.

Factor	Levels	Notes
λ (req/s)	10, 1, 0.1, 0.01, 0.001	open-loop load script
p_1 (error rate)	0.5, 0.1, 0.05	ConfigMap `error_rate`
Guard k	0, 3	H3

Step 1: measure D first (pc1).

Inject `error_rate: 1.0` and record four timestamps per run: ConfigMap applied, first failed response in the load log on the laptop, first nonzero `increase(errors[1m])` in Prometheus, and the rule's pending transition. Ten runs give an empirical distribution for D, which replaces the assumed band in §7.

Step 2: latency grid (laptop drives load, pc1 injects).

Use `scripts/load-open.sh` only, to avoid coordinated omission. Hold λ steady for at least 2.4 h before injection so the 6 h window is full enough—Drill 4 showed this matters. Run one load generator per drill and use a fresh log per run.

Step 3: low-traffic noise (laptop).

Run healthy traffic with `error_rate = 0.001` at each low λ for 7 days per level, or emulate the same request count faster by replaying a synthetic event stream through a separate Prometheus with scaled windows. The second option is cheaper but needs its own validation against one real-time level.

Step 4: bursts (pc1).

Add a `burst_size` field to the fault ConfigMap so the service fails k consecutive requests per failure event, to test the $1/d$ exponent scaling.

Controls.

Freeze rule files during every run (Drill 4 lesson); verify both contact points before starting; log every transition from the Grafana state history API rather than from Discord delivery times; and move the availability SLI to Traefik ingress metrics first, so that requests the application never saw count as failures.

Analysis.

Report per cell the latency distribution against prediction, the residual after subtracting the measured D, and the false-page count with its Poisson interval. Any cell outside its criterion is reported as a falsification, not tuned away.

Discussion and Threats to Validity

The practical message is short: at production traffic the Workbook defaults are close to optimal for their stated policy, and below about 1,000 requests a day the policy itself—not the windows—has to change.

What the model says about current practice.

The 2% and 5% budget-at-detection figures fall out of $B \cdot W_L/T$, so the defaults are internally consistent. What they hide is the pipeline term. A total outage of a 99.5% service burns 1.4% of the budget in pipeline delay alone at $D + \Delta/2 + F = 3$ min, comparable to the policy term, so shaving D (faster config pickup, shorter evaluation intervals, no `for:` on fast-burn rules) buys more than re-tuning windows. Drill 4 also showed the formula's sensitivity to how full the long window is: a 7% traffic gap moved the ticket by 14 minutes.

Internal validity.

The retrospective check uses six events, an assumed D band, and one correction ($g = 0.07$) taken from the drill write-up after the fact. It shows consistency, not predictive power. The prospective protocol fixes this by measuring D independently and pre-registering pass criteria.

Construct validity.

The model treats the estimator as an exact window count. Prometheus `rate()` and `increase()` extrapolate at window edges and depend on scrape alignment, which adds up to one scrape interval of bias; at 15 s against 5 min windows this is under 5%. The latency SLI depends on histogram bucket boundaries, which Drill 3 showed are coarse in Project 1.

External validity.

A two-replica home lab on WSL2 differs from production in scale, traffic shape, and failure modes. Real traffic is diurnal, so λ_{min}, not mean λ, governs noise, and ρ shifts naturally around peaks. Real failures are often correlated across requests (retries, shared dependencies), which the compound-Poisson extension handles only approximately; heavy-tailed burst sizes would weaken the exponent further than $1/d$.

Model limitations.

Incidents are modeled as steps; ramps (a slowly leaking connection pool) detect later than the step formula suggests, by roughly half the ramp duration for a linear ramp. The noise analysis assumes a stationary healthy baseline; a slow drift in b_0 is precisely what the ticket rule is meant to catch, so it is signal, not noise, and lies outside the false-alert framing.

Relation to optimal detectors.

CUSUM on the per-request failure stream would, in principle, dominate the windowed ratio on the latency–noise trade-off. The windowed ratio survives in practice because it is expressible in PromQL, explainable in a runbook, and stateless across restarts. Quantifying how much is lost relative to CUSUM is a natural follow-up.

Conclusion and Future Work

Burn-rate alert configuration can be derived rather than copied. Four results carry the paper:

1. Detection latency is $t = W_L \cdot B/b$ plus pipeline delay, so the product $B \cdot W_L$ fixes both speed and budget-at-detection for every severity (RQ1).

2. False-fire probability per evaluation is bounded by $exp(-\ \lambda W\ \cdot\ KL(\theta\ \|\ p_0))$, which gives a minimum request count per window for a target noise level (RQ2).
3. Below $\lambda_{deg} = 1/(\theta W)$ a single failure trips a window. For the Workbook page rule, false pages peak at about 6 per month near 0.003 req/s, and below about 0.012 req/s any rule with bounded noise is traffic-limited at $k/(\lambda p_1)$ (RQ3).
4. A six-step procedure turns budget, latency, sensitivity, and noise targets into windows, thresholds, and a guard, and states when no rule can work (RQ4).

Future work.

- Run the prospective protocol in §7 and replace the assumed pipeline delay with its measured distribution.
- Extend the analysis to time-slice SLIs, where the unit is a bad minute rather than a bad request.
- Compare the windowed ratio with CUSUM and with Bayesian posterior-budget alerts on the same fault grid.
- Model diurnal traffic explicitly, so that windows or guards can vary with the expected request rate.
- Package the design procedure as a small tool that reads 30 days of Prometheus history and emits recording and alert rules.

Derivation of the False-alert Exponent

Let $E \sim Poisson(a)$ and $S \sim Poisson(c)$ be independent, with $a = \lambda W p_0$ and $c = \lambda W(1 - p_0)$. The window fires when $X = (1 - \theta)E - \theta S > 0$. For $s > 0$, Markov's inequality on e^{sX} gives:

$$Pr(X > 0) \le E[e^{sX}] = exp\Big(a(e^s - 1) + c(e^{-s\theta/(1-\theta)} - 1)\Big).$$

The right side is minimized where $e^s = \theta(1 - p_0) / (p_0(1 - \theta))$. Substituting and simplifying:

$$Pr(X > 0) \le exp\Big(-\ \lambda W\ \cdot\ KL(\theta\ \|\ p_0)\Big).$$

The bound is valid for $\theta > p_0$ and tight in exponent as $\lambda W \to \infty$.

Simulation Code

The Monte Carlo in §6 simulates one-second bins: $N \sim Poisson(\lambda)$, $E \sim Binomial(N, p(t))$, cumulative sums give window counts, and the rule is evaluated every 60 s at a uniformly random phase. Latency runs use 200 incidents per point after a one-window warm-up; noise runs use 24 simulated months per rate below 0.03 req/s. Seeds 7 and 11 (NumPy `default_rng`). The script will be committed to the Project 1 repository under `observability/analysis/`.

99

B. Beyer, C. Jones, J. Petoff, N. R. Murphy (eds.). *Site Reliability Engineering: How Google Runs Production Systems*. O'Reilly, 2016. Ch. 4, "Service Level Objectives."

B. Beyer, N. R. Murphy, D. K. Rensin, K. Kawahara, S. Thorne (eds.). *The Site Reliability Workbook*. O'Reilly, 2018. Ch. 5, "Alerting on SLOs."

W. A. Shewhart. *Economic Control of Quality of Manufactured Product*. Van Nostrand, 1931.

E. S. Page. Continuous inspection schemes. *Biometrika*, 41(1/2):100–115, 1954.

G. Lorden. Procedures for reacting to a change in distribution. *Annals of Mathematical Statistics*, 42(6):1897–1908, 1971.

H. Chernoff. A measure of asymptotic efficiency for tests of a hypothesis based on the sum of observations. *Annals of Mathematical Statistics*, 23(4):493–507, 1952.

Prometheus. Query functions `rate()` and `increase()`, and alerting rules (`for` clause). https://prometheus.io/docs/, 2024.

S. Babu. Project 1 repository and drill write-ups, drills/001–005 and `SLO.md`: homelab-slo-platform. GitHub, 2026.